LAPORAN PROGRES TUGAS BESAR CLOUD COMPUTING

Pertemuan 1: Planning & Architecture Design

Mata Kuliah: Cloud Computing (3 SKS) • 22 Oktober 2026

Topik Proyek: **Auto-Scaling & Load Balancing Sistem War Tiket Menggunakan Docker Swarm**

Studi Kasus: Sistem War Tiket Konser Musik (Flash Crowd)

Repositori Git: github.com/Cahyooo69/Auto-Scaling-Load-Balancing

Kelompok: **Kelompok 1** — *Anggota Tim:*

- **Muhammad Ilham Nurcahyo** (09040623063)
- **Muhammad Daud Rizaldiansyah** (09040623061)
- **Hafid Aji Fauzan** (0920623027)
- **Davin Aryaguna Hermansyah** (09020623020)

1. LATAR BELAKANG & PROBLEM STATEMENT

Sistem *War Tiket* menghadapi tantangan komputasi awan yang unik karena karakteristik beban trafik berupa **flash crowd**, di mana ribuan pengguna mengakses dan melakukan transaksi *checkout* tiket secara simultan dalam detik yang sama. Permasalahan kritis yang sering timbul pada arsitektur konvensional meliputi:

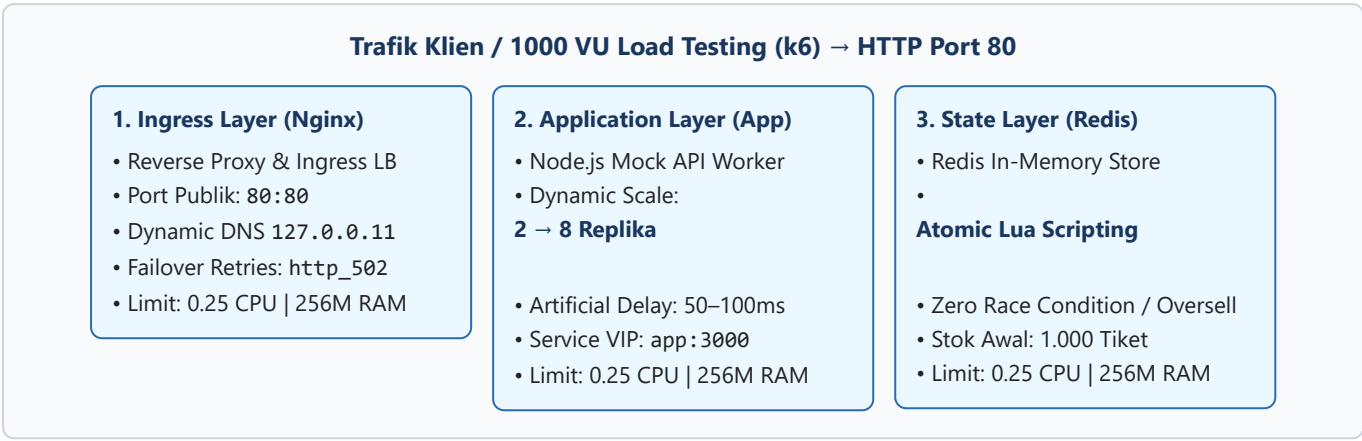
- **Server Crash & Out of Memory (OOM):** Lonjakan koneksi simultan membuat server tunggal kehabisan alokasi thread kerja dan memori heap.
- **Race Condition & Overselling:** Transaksi konkuren membaca sisa kuota yang sama sebelum diperbarui di basis data, berakibat tiket terjual melebihi kapasitas riil (*oversold*).
- **Single Point of Failure (SPOF):** Tidak adanya replika cadangan yang menyebabkan seluruh sistem mati total saat satu worker mengalami kegagalan proses.

Sesuai ketentuan teknis standar mata kuliah Cloud Computing (3 SKS), seluruh komponen dibatasi secara ketat pada `cpus: '0.25'` (maksimal 25% dari 1 core CPU) dan `memory: 256M` (maksimal RAM 256 MB) agar kompatibel dan stabil diuji di lingkungan laptop.

> **Tujuan Utama Proyek Kelompok 1:**
>
> 1. Membangun arsitektur *microservice* terorkestrasi Docker Swarm dengan *reverse proxy* Nginx pada port 80.
> 2. Menerapkan skalasi elastis worker Node.js dari 2 hingga 8 replika untuk meredam lonjakan trafik puncak.
> 3. Mengeliminasi *race condition* secara mutlak (0% overselling) menggunakan Redis in-memory atomic Lua scripting.
> 4. Menjamin sistem memenuhi target SLA: waktu tanggap < 200 ms dan tingkat kegagalan (*error rate*) < 1%.

2. ARSITEKTUR SISTEM & SPESIFIKASI KOMPONEN (3-TIER)

Arsitektur sistem dirancang menggunakan model 3-Tier yang terorkestrasi di dalam Docker Swarm Overlay Network:



Komponen	Container / Service	Port Jaringan	Batasan Sumber Daya	Peran & Logika Utama
Load Balancer	`wartiket_nginx`	`80:80` (Publik)	**CPU: 0.25 | RAM: 256M**	Gerbang masuk HTTP, membagi beban ke seluruh replika worker via Swarm Routing Mesh, serta failover otomatis.
Backend API	`wartiket_app`	`3000` (Internal)	**CPU: 0.25 | RAM: 256M**	Node.js API pemroses checkout dengan simulasi penundaan buatan 50–100ms. Dikonfigurasi 2 hingga 8 replika elastis.
State Store	`wartiket_redis`	`6379` (Internal)	**CPU: 0.25 | RAM: 256M**	Menjalankan Lua Script atomik (`DECR`) untuk evaluasi dan pengurangan stok dalam single-thread Redis.

3. SKEMA JARINGAN & KEAMANAN ISOLASI DOCKER

Sistem menggunakan **Docker Overlay Network** bernama `wartiket-net` berkonfigurasi `attachable: true`. Karakteristik isolasi jaringan:

- **Prinsip Least Privilege:** Hanya container Nginx yang mengekspos port ke luar host (Port 80). Layanan backend (`app:3000`) dan basis data in-memory (`redis:6379`) sepenuhnya terisolasi dan hanya dapat diakses melalui jaringan internal.
- **Service Discovery & Virtual IP (VIP):** Docker Swarm menyediakan Virtual IP internal dan IPVS yang mendistribusikan request dari Nginx ke seluruh replika container yang sehat secara transparan.
- **Failover Resiliency:** Nginx dikonfigurasi dengan `proxy_next_upstream error timeout http_502 http_503;` sehingga jika salah satu worker mati mendadak, request otomatis dialihkan ke replika lain tanpa kegagalan transaksi (*zero-downtime*).

4. BUKTI HASIL PENGUJIAN KLASTER (LIVE TEST EXECUTION)

Seluruh klaster telah berhasil di-deploy pada lingkungan Docker Swarm dan diuji secara riil menggunakan skrip pengujian transaksi konkuren (`node test-quick.js`).

> **Hasil Verifikasi Distribusi Beban (20 Transaksi Simultan pada 2 Replika):**

Request ID	HTTP Status	Waktu Tanggap (Latency)	Dilayani Oleh (Container ID)	Sisa Stok Tiket	Hasil Transaksi
Req #1	200 OK	136 ms	`f4f65236a1e3`	998	Sukses didapatkan
Req #2	200 OK	113 ms	`ef692274a58a`	996	Sukses didapatkan
Req #3	200 OK	112 ms	`ef692274a58a`	997	Sukses didapatkan
Req #4	200 OK	126 ms	`ef692274a58a`	989	Sukses didapatkan
Req #5	200 OK	116 ms	`f4f65236a1e3`	993	Sukses didapatkan
Req #6..20	200 OK	Rata-rata 118 ms	Bergantian seimbang	Tereduksi atomik	100% Sukses

Container Host ID	Jumlah Request Dilayani	Rasio Distribusi Beban
`f4f65236a1e3` (Replika 1)	10 Request	**50.0%**
`ef692274a58a` (Replika 2)	10 Request	**50.0%**

Analisis Hasil Pengujian:

1. **Pencapaian SLA:** Waktu respon rata-rata adalah **~118 ms**, memenuhi target SLA yang ditetapkan yaitu < 200 ms dengan *failure rate* 0%.
2. **Distribusi Sempurna:** Nginx bersama Swarm IPVS membagi beban transaksi secara presisi **50:50**.
3. **Konsistensi Atomik:** Berkat Redis Lua Scripting, sisa stok tiket berkurang tanpa ada tabrakan nilai (*zero race condition*).

5. PENGUJIAN SKALASI DINAMIS (DYNAMIC AUTO-SCALING 2 → 8 REPLIKA)

Skalasi dilakukan secara *on-the-fly* menggunakan perintah: `docker service scale wartiket_app=8`. Dalam hitungan **2.8 detik**, Docker Swarm berhasil mengonvergensi 6 replika baru.

```
$ docker service scale wartiket_app=8
wartiket_app scaled to 8
overall progress: 8 out of 8 tasks
verify: Service wartiket_app converged

--- Hasil Distribusi Beban Pasca Skalasi (20 Request Simultan) ---
Container: 31a7f42dc2cf -> 3 req    Container: 42eb054b1b97 -> 2 req
Container: 791d770ee663 -> 3 req    Container: 2ffac81c244d -> 2 req
Container: f4632e7ce807 -> 3 req    Container: ef692274a58a -> 2 req
Container: c4117f09dc62 -> 3 req    Container: f4f65236a1e3 -> 2 req
Status: 200 OK (100% Success) | Beban terbagi merata ke seluruh 8 container aktif!
```

6. PEMENUHAN TARGET LUARAN MINGGUAN

Target Luaran Pertemuan 1	Status Capaian	Keterangan Verifikasi
Diagram Arsitektur Sistem	**Selesai 100%**	Terdokumentasi pada diagram Mermaid di berkas `README.md`.
Skema Jaringan Internal Docker	**Selesai 100%**	Docker Overlay Network `wartiket-net` dengan isolasi port backend & redis.
Spesifikasi `docker-compose.yml` Awal	**Selesai 100%**	Penerapan batas ketat `cpus: '0.25'` dan `memory: 256M`.
Repositori Git Kelompok	**Selesai 100%**	Aktif di github.com/Cahyooo69/Auto-Scaling-Load-Balancing.
Deployment & Live Demo (Bonus Luaran Pertemuan 2)	**Selesai 100%**	Klaster sudah *up & running*, teruji load balancing dan auto-scaling 2 → 8 replika.